

What does *ko.applyBindings* do?

The first parameter says what view model object you want to use with the declarative bindings it activates.

Optionally, you can pass a second parameter to define which part of the document you want to search for data-bind attributes. For example:

```
ko.applyBindings(myViewModel, document.
getElementById(elementId))
```

This restricts the activation to the element with ID *elementId* and its descendants, which is useful if you want to have multiple view models and associate each with a different region of the page.

Observables

One of the key benefits of KO is that it updates the UI automatically when the view model changes. In order for KO to know when parts of your view model change, declare your model properties as observables, because these are special JavaScript objects that can notify subscribers about changes, and can automatically detect dependencies.

For example, rewrite the preceding view model object as follows:

```
var employeeModel={
name:ko.observable('Anna'),
department:ko.observable('Sales')
};
```

You don't have to change the view at all -- the same data-bind syntax will keep working. The difference is that it's now capable of detecting changes, and when it does, it will update the view automatically.

Observables can be set up on a collection as well. These are called observable arrays. Similarly, KO has the concept of computed observables which are functions that are dependent on one or more other observables, and will automatically update whenever any of these dependencies change.

For example, given the following view model class...

```
function AppViewModel() {
this.firstName = ko.observable('Bob');
this.lastName = ko.observable('Smith');
}
```

...you could add a computed observable to return the full name, as follows:

```
function AppViewModel() {
// ... leave firstName and lastName
unchanged ...
this.fullName =
ko.computed(function() {
return this.firstName() + " " +
this.lastName();
}, this);
}
```

Now you could bind UI elements to it, as follows:

```
The name is <span data-bind="text:
fullName"></span>
```

...and they will be updated whenever the *firstName* or *lastName* changes (your evaluator function will be called once each time any of its dependencies change, and whatever the value you return will be passed on to observers such as UI elements or other computed observables).

Dependency tracking

Now that we know that there are observables and computed observables, let us look at how the dependency tracking works in Knockout at a high level.

The tracking algorithm goes like this:

1. Whenever you declare a computed observable, KO immediately invokes its evaluator function to get its initial value.
2. While the evaluator function is running, KO sets up a subscription to any observable (including other computed observables) that the evaluator reads. The subscription callback is set to cause the evaluator to run again, looping

the whole process back to Step 1 (disposing of any old subscriptions that no longer apply).

3. KO notifies subscribers about the new value of the computed observable.

So, Knockout doesn't just detect dependencies the first time the evaluator runs — it re-detects them every time. This means, for example, that the dependencies can vary dynamically: Dependency A could determine whether the computed observable also depends on B or C. Then, it will only be re-evaluated when either A or your current choice of B or C changes. You don't have to declare dependencies — they're determined at runtime from the code's execution. If the evaluator doesn't access any observables, the computed observable will have no dependencies and won't ever need to call the evaluator function again. In that case, to save resources, the computed observable will be automatically 'disposed' of.

The other neat trick is that declarative bindings are simply implemented as computed observables. So, if a binding reads the value of an observable, it becomes dependent on that observable, which causes that binding to be re-evaluated if the observable changes.

This article covers the basics of KO for beginners. <https://knockoutjs.com> offers in-depth documentation as well as interactive tutorials for a better understanding. From my personal experience of working with KO, I would say that it can get pretty tricky when you have complex bindings and computations in place. Templating and custom binding can help save time and make things more standardised. **END** 🐧

Reference

<https://knockoutjs.com>

By: Athira Lekshmi

The author is an open source enthusiast.